

Evaluating Spoken Reading Comprehension Answers via Automatic Speech Recognition and Semantic Similarity

Ava Jalalimoghaddam
University of Gothenburg
gussamzo@student.gu.se

Abstract

Reading comprehension is typically assessed using multiple-choice questions, which limit how freely students can express understanding. We present a pipeline for automatically evaluating short spoken answers to reading comprehension questions using automatic speech recognition (Whisper) and semantic similarity (Sentence-BERT), rather than exact-text matching. We construct a 50-question evaluation set from SQuAD v1.1, generate spoken answers via text-to-speech, and evaluate a hybrid scoring approach that combines semantic similarity with exact numeric matching to address a limitation we discover in the base similarity model: it fails to distinguish factually different numeric answers that share similar sentence structure. Our system achieves 96% accuracy against human-labeled ground truth, with the majority of remaining errors attributable to speech recognition rather than semantic scoring. We additionally report a brief fine-tuning experiment on Sentence-BERT and discuss its limitations given the small scale of available labeled data.

1 Introduction

Reading comprehension assessment commonly relies on multiple-choice questions, which are easy to grade automatically but constrain how freely a student can express understanding, sometimes allowing correct answers to be guessed rather than reasoned. Free-form spoken answers offer a more authentic test of comprehension, but introduce two compounding challenges: the same correct meaning can be expressed in many different ways, and automatic speech recognition (ASR) introduces transcription noise on top of this natural variation. Existing automatic grading systems that rely on exact-text matching are poorly suited to this setting, as they penalize correct answers that use different wording than the reference.

This project investigates whether combining ASR with semantic similarity scoring can reliably judge the correctness of short spoken answers to reading comprehension questions, while remaining robust to the kind of wording variation and transcription noise found in realistic use. We build a full pipeline: Whisper (Radford et al., 2022) transcribes spoken answers, and Sentence-BERT (Reimers and Gurevych, 2019) embeddings are compared via cosine similarity to a reference answer drawn from the Stanford Question Answering Dataset (SQuAD) (Rajpurkar et al., 2016). In developing this pipeline, we identify a specific failure mode of semantic similarity scoring on numeric answers, and propose and evaluate a lightweight hybrid scoring method to address it.

Our contributions are: (1) an evaluation methodology combining ASR and semantic similarity for spoken short-answer grading; (2) an empirical demonstration that plain semantic similarity is unreliable for numeric answers, along with a hybrid fix; (3) a threshold-selection procedure grounded in an explicit precision/recall trade-off analysis; and (4) an interactive demo and an error analysis characterizing the remaining sources of error in the system.

2 Related Work

Automatic Short Answer Grading (ASAG) has a long history of using semantic similarity to compare student responses against reference answers, motivated by the observation that exact keyword or string matching penalizes correct answers phrased differently than expected (Mohler and Mihalcea, 2009). Early approaches relied on hand-crafted features: corpus-based similarity measures such as Latent Semantic Analysis, string-similarity metrics, and syntactic alignment features (Gomaa and Fahmy, 2012). Sultan et al. (2016) combined multiple lexical and semantic features (word align-

ment, vector similarity) to achieve strong correlation with human grades while remaining computationally lightweight. More recent work has shifted toward neural sentence embeddings, following the introduction of transformer-based models such as Sentence-BERT (Reimers and Gurevych, 2019), which produce fixed-size vector representations optimized so that semantically similar sentences have high cosine similarity — directly applicable to comparing a student’s answer against a reference.

Our work differs from this line of research in two respects. First, prior ASAG work operates on *written* student answers; we extend the task to *spoken* answers, introducing automatic speech recognition as an additional pipeline stage and, correspondingly, an additional source of noise to remain robust to. Second, in developing our pipeline we identify and address a specific limitation of using a general-purpose sentence embedding model for grading: its inability to reliably verify numeric correctness (Section 4.2), which to our knowledge is not explicitly addressed in the ASAG literature we reviewed, where reference answers are often conceptual rather than numeric.

3 Data

We use the Stanford Question Answering Dataset (SQuAD) v1.1 (Rajpurkar et al., 2016), a collection of approximately 100,000 question-answer pairs derived from Wikipedia articles, each consisting of a passage, a question, and a reference answer. We parse the nested JSON release into a flat table of passage/question/answer triples for both the train and development splits.

From the development split, we manually construct a 50-question evaluation set, stratified by reference answer length: 15 short answers (1 word), 25 medium answers (2–4 words), and 10 long answers (5+ words), with questions sampled for topical and question-type variety (who/what/when/where/how many). This stratification ensures the evaluation set is not dominated by the short factual answers that make up the majority of SQuAD, and allows us to probe system behavior across answer types.

Since SQuAD provides only written reference answers, we synthesize spoken versions using two text-to-speech (TTS) engines: gTTS (an online, more natural-sounding engine) and pyttsx3 (an offline engine using Windows system voices), with 25 answers generated by each, to introduce some

Question	Answer
When is the Evolution Festival hosted?	May
What stadium did Super Bowl 50 take place in?	Levi’s Stadium
How large are Plastoglobuli?	45–60 nanometers across

Table 1: Representative examples from the evaluation set (short, medium, long answer types).

diversity in the audio source. The stratification procedure worked as follows: reference answers in the development split were bucketed by word count into short (1 word, $n = 3,428$ candidates), medium (2–4 words, $n = 5,421$), and long (5+ words, $n = 1,720$) categories. From each bucket, a random pool of 30 candidates was sampled, from which 15, 25, and 10 questions respectively were manually selected for topical and question-type diversity. Table 1 shows representative examples from the final evaluation set.

4 Methods

4.1 Pipeline Overview

Our system evaluates a spoken answer in three stages. First, the answer audio is transcribed using Whisper (Radford et al., 2022), specifically the tiny model variant run via faster-whisper on CPU, chosen to keep iteration fast during development. Second, the resulting transcript is compared to the reference answer using Sentence-BERT (Reimers and Gurevych, 2019) (the all-MiniLM-L6-v2 model), which embeds both texts and computes their cosine similarity. Third, the similarity score is mapped to a three-way label — Correct, Incorrect, or Borderline — via a threshold selected empirically (Section 5).

4.2 A Failure Mode in Numeric Answers

Preliminary experiments comparing semantic similarity across paraphrases, unrelated answers, and factually-wrong “near-miss” answers (same sentence structure, different critical detail) revealed that Sentence-BERT does not reliably distinguish numeric correctness from topical similarity. For example, “45–60 nanometers across” and “20–30 nanometers across” — factually different measurements — scored a cosine similarity of 0.943, higher than several genuinely correct paraphrases in our test set (which ranged 0.552–0.796). We attribute this to the model weighting overall sentence structure and shared vocabulary rather than verifying

that numeric tokens match.

4.3 Hybrid Similarity Scoring

To address this, we implement a hybrid scoring function that supplements semantic similarity with explicit numeric verification. Numbers are extracted from both the reference answer and the transcript using a regular expression, with a targeted normalization step that removes thousands-separator commas (matching the pattern of a digit, comma, and exactly three digits) without merging distinct comma-separated numbers. If the reference answer contains no numbers, the semantic similarity score is returned unchanged. If it does contain numbers, an exact match (as a set, so order is irrelevant) between the reference and transcript numbers is required: on a match, the score is set to a fixed 1.0 (rather than the raw semantic score, which we found to be unreliable even for numerically-identical short strings, e.g. “1893” vs. “1,893” scored only 0.193 in raw cosine similarity); on a mismatch, the score is forced to 0, regardless of the semantic similarity. This deliberately strict penalty was chosen over a partial-credit scheme for simplicity, given the scope of this project.

4.4 Model Selection

We use the tiny variant of Whisper, the smallest available model size, running via the faster-whisper implementation on CPU with int8 quantization. This choice trades transcription accuracy for speed: on CPU, larger Whisper variants (e.g., small, medium) can take 10–30 seconds or more per short audio clip, compared to roughly 1–2 seconds for tiny, which was necessary for iterating quickly across 50 evaluation examples during development. We note this as a deliberate speed/accuracy trade-off appropriate to this project’s scope (see Limitations); a production deployment would likely benefit from a larger model.

For semantic similarity, we use all-MiniLM-L6-v2, a lightweight (22M parameter) Sentence-BERT model producing 384-dimensional embeddings, chosen for its balance of inference speed and quality on CPU.

4.5 Spoken Answer Generation

Since SQuAD provides only text, spoken versions of the 50 evaluation answers were synthesized rather than recorded from human speakers, for practical reasons discussed in Limitations section. To

introduce some acoustic diversity despite using synthetic speech, answers were split evenly between two text-to-speech engines: gTTS (Google’s online text-to-speech service, producing comparatively natural-sounding speech) and pyttsx3 (an offline engine using the local operating system’s speech synthesis voices, producing more robotic-sounding output). The 50 answers were shuffled and split into two balanced groups of 25 prior to assignment, to avoid confounding TTS engine with answer length or type. A post-hoc comparison (Section 5.4) found no substantial difference in downstream transcription accuracy between the two engines (gTTS: 21 correct / 3 incorrect / 1 borderline; pyttsx3: 20 correct / 4 incorrect / 1 borderline out of 25 each).

4.6 Numeric Extraction Algorithm

The following pseudocode describes the number-extraction procedure used by the hybrid scoring function (Section 4.2). The critical design detail is the comma-normalization step: a naive approach of stripping all commas before extracting digit sequences would incorrectly merge distinct comma-separated numbers (e.g., “1,2,3” would become the single number 123). Instead, we only remove commas that appear in the thousands-separator position — a digit, followed by a comma, followed by exactly three digits — leaving other comma usage intact.

```
function EXTRACTNUMBERS(text):  
    text ← replace pattern  
    (digit)(comma)(3 digits) with (digit)(3  
    digits)  
    numbers ← all maximal digit  
    sequences in text  
    return numbers as a set
```

5 Results

5.1 Threshold Selection

To select a decision threshold, we manually labeled all 50 evaluation examples as correct or incorrect based on independent human judgment of the transcript against the reference answer (without reference to the system’s similarity score, to avoid circular evaluation). We then evaluated six candidate thresholds against these labels (Table 2).

Higher thresholds improve precision at the cost of recall, since stricter cutoffs increasingly reject correct answers whose raw semantic similarity falls below the threshold. We select **0.60** as the final

Threshold	Acc.	Prec.	Rec.
0.60	0.960	0.977	0.977
0.70	0.920	0.976	0.930
0.75	0.900	0.975	0.907
0.80	0.880	0.974	0.884
0.85	0.880	1.000	0.860
0.90	0.840	1.000	0.814

Table 2: Accuracy, precision, and recall at each candidate threshold.

threshold, as it achieves the highest accuracy and the best balance between precision and recall (both ≈ 0.977). We weigh the two error types explicitly: a false rejection (marking a correct answer wrong) unfairly penalizes an individual student, while a false acceptance (marking a wrong answer correct) undermines the grading system’s overall validity and could indirectly disadvantage other students (e.g., under comparative or curve-based grading); we judge the former as somewhat more directly unfair, favoring the threshold that avoids sacrificing recall.

5.2 Threshold Selection Methodology

The threshold-tuning process proceeded in three stages. First, all 50 pipeline outputs (reference answer, Whisper transcript, and raw similarity score) were reviewed manually, with correctness judged independently of the displayed similarity score to avoid circular evaluation — that is, the human judged whether the transcript conveyed the same meaning as the reference answer, without being influenced by the system’s own confidence. Second, six candidate thresholds spanning the plausible range (0.60 through 0.90, chosen to bracket the similarity scores observed for genuine paraphrases in our preliminary experiments) were evaluated against these labels using standard binary classification metrics. Third, the final threshold was selected not by accuracy alone, but by explicitly weighing the practical consequences of the two possible error types for a grading application, as discussed above.

One subtlety in this process concerns proper nouns with imperfect phonetic transcription (e.g., “Pleurobrachia” transcribed as “Plorobracia”). During manual labeling, the human annotator was genuinely uncertain how to treat such cases: they represent an ASR mishearing of a correct spoken answer rather than a wrong answer, but the transcribed string differs enough from the reference that a stricter grader might disagree. We adopted

Metric	Score
Accuracy	0.960
Precision	0.977
Recall	0.977
F1	0.977

Table 3: Final evaluation metrics.

	Pred. Incorrect	Pred. Correct
Actual Incorrect	6	1
Actual Correct	1	42

Table 4: Confusion matrix (50 examples).

a consistent policy of labeling such near-phonetic-match proper nouns as correct, reasoning that a human grader listening to the same audio would recognize the intended word. This policy choice affects only 2 of the 50 examples but illustrates a genuine source of subjectivity in what otherwise appears to be an objective evaluation procedure.

5.3 Final Evaluation

Table 3 reports the final evaluation metrics using the selected threshold (Borderline predictions, discussed below, are counted as Correct).

To capture genuine ambiguity, we additionally define a Borderline band of ± 0.05 around the threshold (0.55–0.65). Of the 50 examples, 41 were classified Correct, 7 Incorrect, and 2 Borderline. Notably, the two Borderline cases — “Pleurobrachia”/“Plorobracia” and “William Stranahan”/“William Strenohin” — were exactly the two cases the human annotator was independently uncertain about during manual labeling, suggesting the Borderline band captures genuine ambiguity rather than an arbitrary numeric cutoff.

Table 4 shows the confusion matrix against human labels.

Table 5 shows representative predictions across the three outcome categories, illustrating typical behavior: near-exact transcriptions score highly, severe ASR failures score near zero, and the two Borderline cases (discussed above) fall in an intermediate range reflecting genuine uncertainty.

5.4 Error Analysis

We categorized all 9 non-Correct predictions by root cause. Eight of nine (89%) were attributable to ASR mistranscription — primarily on rare proper nouns and specialized terminology (e.g., “Beyoncé” transcribed as “Be on say”; “Nintendo” tran-

Answer	Transcript	Sim.	Pred.
the Queen	The Queen.	0.969	Correct
Beyoncé	Be on say.	0.168	Incorrect
10,000 m2	10,000 square meters.	0.000	Incorrect
Pleurobrachia	Plorobracia	0.640	Borderline

Table 5: Representative predictions across outcome categories.

Pair	Pre	FT
“1852”/“1852”	0.625	0.627
“Paris”/“Paris.”	0.963	0.968
Lorraine example	0.279	0.261
Gas turbines example	0.160	0.144

Table 6: Pretrained vs. fine-tuned similarity scores.

scribed to non-Latin script). Only one error (11%) stemmed from the scoring pipeline itself: the reference answer “10,000 m2” was correctly transcribed as “10,000 square meters” by Whisper, but our numeric-matching logic treated the “2” in “m2” as a standalone number to match, causing a false rejection despite the transcript being semantically correct. This indicates that, on our evaluation set, speech recognition quality is the primary bottleneck on system accuracy, not the semantic similarity or hybrid scoring components.

5.5 Fine-Tuning Experiment

To satisfy the requirement of demonstrating a working training procedure, we fine-tuned the Sentence-BERT model on our 50 labeled examples (answer/transcript pairs with the human correctness label as target similarity), using CosineSimilarityLoss for one epoch (training completed in under 3 seconds). Comparing the fine-tuned model against the original on held-out pairs showed only negligible, inconsistent changes in similarity (within ± 0.02 in either direction; Table 6), which we report as an honest null result: 50 examples and one epoch are insufficient to meaningfully shift the weights of a model pretrained on millions of sentence pairs. This confirms that the system’s accuracy derives from the pretrained model’s general-purpose semantic understanding combined with the hybrid numeric-matching logic, not from task-specific fine-tuning.

6 Discussion

Our results suggest that combining a pretrained ASR model with pretrained sentence embeddings,

augmented with a small amount of task-specific logic (numeric matching), can achieve strong performance on spoken short-answer grading without any task-specific model training. The error analysis further suggests that improving speech recognition quality — particularly on rare proper nouns, which accounted for most remaining errors — would yield larger accuracy gains than further tuning of the semantic similarity component.

The numeric-matching fix illustrates a broader point: general-purpose semantic similarity models are not designed to verify factual correctness of specific tokens (numbers, dates), only overall topical and structural similarity. Systems relying on such models for factual short-answer grading should consider supplementary exact-match verification for answer types where a specific value, rather than a paraphrasable concept, constitutes correctness.

Beyond the numeric-matching fix, our error analysis (Section 5.4) has a further practical implication: since 89% of errors originated in speech recognition rather than semantic scoring, future effort aimed at improving system accuracy should prioritize ASR quality — for instance, using a larger Whisper variant, or incorporating a pronunciation dictionary or domain-specific vocabulary list for rare proper nouns — over further refinement of the similarity scoring logic. This has a practical implication: it is easy to assume that improving the scoring algorithm is the best way to improve accuracy. However, our results show that improving speech recognition would likely help more, since most errors come from that stage, not from the scoring stage.

We also note that our threshold selection (Section 5) reflects an explicit value judgment about the relative cost of two error types, rather than a purely statistical optimum. This is a deliberate design choice: automated grading systems inevitably encode some notion of acceptable error trade-offs, and we believe this should be made explicit and justified rather than left implicit in a default choice of metric (e.g., simply maximizing accuracy without considering which direction of error is being traded off). Practitioners deploying similar systems should consider their own context-specific costs — for example, a high-stakes summative exam may warrant a stricter threshold (favoring precision) than a low-stakes formative practice tool (where favoring recall, and therefore not discouraging students who answered correctly, may be more

valuable).

Finally, our fine-tuning null result (Section 5.5) is itself a useful data point for anyone considering a similar project: it demonstrates that for small-scale spoken-answer grading tasks, the marginal benefit of fine-tuning a general-purpose sentence embedding model on a handful of labeled examples is likely to be negligible, and that data collection effort might be better spent on expanding the human-labeled evaluation set (to enable more robust threshold selection) than on assembling a fine-tuning dataset of similar size.

7 Conclusion

We built and evaluated a pipeline for automatically grading spoken reading comprehension answers using ASR and semantic similarity, achieving 96% accuracy against human judgment on a 50-question evaluation set. We identified and fixed a specific failure mode of semantic similarity scoring on numeric answers, and showed through error analysis that the dominant remaining source of error is speech recognition rather than semantic scoring. A brief fine-tuning experiment on our 50-example dataset yielded a null result, which we interpret as expected given this small scale, rather than as evidence that fine-tuning is generally ineffective for this task. With a substantially larger labeled dataset, fine-tuning could plausibly yield meaningful improvements; our result only establishes that 50 examples and one epoch are insufficient, not that fine-tuning has no value at scale. This supports a narrower conclusion: on the data available to this project, the system’s strong performance is attributable to pretrained model quality plus lightweight task-specific logic, rather than to fine-tuning.

8 Implementation and Software Engineering Considerations

Beyond the core NLP pipeline, several implementation decisions affect the reliability and maintainability of the system, which we document here for completeness.

8.1 Modular Pipeline Design

The system is organized as a set of independent, reusable scripts corresponding to each pipeline stage: dataset loading and flattening, evaluation set construction, batch transcription, and hybrid similarity scoring. Each stage reads from and writes

to intermediate CSV files, allowing any stage to be re-run independently without recomputing earlier stages — for example, re-running the scoring stage after modifying the hybrid similarity function does not require re-transcribing all 50 audio files. This design also makes each stage independently testable and inspectable, which was valuable during development for isolating the source of unexpected behavior (e.g., distinguishing whether an anomalous score originated from a transcription error or a scoring logic error).

8.2 Expensive Resource Loading

Both the Whisper and Sentence-BERT models are loaded once per script execution rather than per function call, since model loading is comparatively expensive relative to a single inference call. This pattern matters at the scale of processing 50 audio files: repeated model loading would have made iterative development substantially slower, though it does not affect the pipeline’s correctness.

8.3 Interactive Demonstration

To validate the pipeline under realistic, real-time conditions rather than only on pre-recorded evaluation data, we built an interactive demonstration using Gradio, allowing a user to select a question, record a spoken answer via microphone, and view the transcript, similarity score, and prediction live. This surfaced at least one implementation issue not visible when testing on pre-recorded files: the audio-recording component initially returned no audio data due to a combination of the demo being tested inside an embedded browser context (which does not reliably grant microphone access) and a missing explicit audio format specification. Building and debugging this interactive component was valuable independent of its use as a deliverable, since it forced end-to-end validation of the pipeline on genuinely novel input rather than only the fixed evaluation set used throughout the rest of this report.

9 Development Environment and Practical Challenges

Several practical software engineering challenges arose during development that, while not central to the NLP methodology, consumed significant development effort and are documented here both for completeness and as a resource for others attempting similar projects.

9.1 Environment Configuration

Running Jupyter notebooks with a project-specific virtual environment on Windows required explicitly registering the virtual environment as a Jupyter kernel; without this step, notebooks silently executed against a pre-existing Anaconda Python installation rather than the project’s own environment, causing packages installed in the virtual environment to appear unavailable despite having been correctly installed. This was diagnosed by comparing `sys.executable` (which revealed the notebook was running Anaconda’s Python) against the expected virtual environment path, and resolved using the `ipykernel` package to register a project-specific kernel.

9.2 Working Directory Inconsistencies

Because Jupyter notebooks resolve relative file paths against the directory from which the notebook server was launched (which varied across sessions depending on how Jupyter was started), file-not-found errors recurred throughout development whenever a notebook attempted to read from or write to the `data/` directory using a hardcoded relative path. This was resolved by consistently checking `os.getcwd()` before diagnosing path-related errors, rather than assuming a fixed relative path would work across all execution contexts.

9.3 Version Control of Large Files

An initial commit inadvertently included the project’s virtual environment directory (containing several hundred megabytes of PyTorch binaries) before a `.gitignore` file had been created, causing GitHub to reject the push due to file size limits. This was resolved by discarding the git history entirely and re-initializing the repository with a correctly configured `.gitignore` in place from the first commit, since the erroneous history contained no content worth preserving. This underscores the importance of establishing exclusion rules for large, regenerable artifacts (virtual environments, downloaded datasets, model weights) before the first commit rather than after.

9.4 Audio Processing Edge Cases

Two distinct audio-related edge cases were encountered. First, a copy-paste error while downloading the two SQuAD data splits resulted in both files containing identical (dev-split) content, detected via an unexpectedly identical file size before either file had been opened, and resolved by

re-downloading each file individually while explicitly verifying the URL argument. Second, integrating microphone recording into the Gradio-based demonstration failed silently (returning no audio data) due to a combination of testing within an embedded editor browser context and an unspecified audio encoding format, isolated through a minimal reproduction consisting of a single-purpose test application before being resolved in the main application.

Limitations

Our evaluation set contains only 50 manually-labeled examples; while sufficient for methodology development, differences between candidate thresholds correspond to only 2–3 disagreements at this scale, limiting the statistical robustness of the threshold selection. All spoken answers were synthesized via text-to-speech rather than recorded from real speakers, so error patterns characteristic of human speech (disfluencies, accents, background noise) are not represented. The hybrid numeric-matching logic has a known edge case: unit abbreviations containing digits (e.g., “m2” for square meters) can be misidentified as standalone numbers, causing false rejections; this occurred in 1 of 50 examples. Finally, our fine-tuning experiment used a dataset far too small to draw conclusions about whether task-specific fine-tuning could improve performance at scale.

10 Reproducibility

All code, data processing scripts, and instructions for reproducing this work — including the dataset download, evaluation set construction, TTS generation, transcription, hybrid scoring, evaluation, fine-tuning, and interactive demo — are available in the project repository (Appendix A). The fine-tuning script (Section 5.5) can be run end-to-end on the provided 50-example dataset in under 3 seconds on CPU, requiring no GPU or external API access beyond the one-time download of pretrained model weights.

Ethical Considerations

Automated grading systems carry risk of harm when deployed in real educational settings: a false rejection (Section 5) could unfairly lower a student’s grade, and our error analysis shows this risk is concentrated on speakers whose pronunciation of rare or unfamiliar words diverges from ASR

training data, potentially disadvantaging non-native speakers or speakers of underrepresented accents and dialects, since Whisper’s training data and Sentence-BERT’s training corpus are not equally representative of all speech varieties. Because our evaluation set uses synthetic (text-to-speech) rather than human speech, we cannot directly assess this risk on real speakers, and we consider this an important limitation for any real-world deployment: further evaluation on speech from diverse speaker populations would be necessary before using a system like ours for actual grading decisions, and we recommend that any deployment retain human oversight (e.g., treating Borderline and low-confidence predictions as requiring manual review) rather than fully automating high-stakes grading.

References

- Wael H Gomaa and Aly A Fahmy. 2012. Short answer grading using string similarity and corpus-based similarity. *International Journal of Advanced Computer Science and Applications*, 3(11).
- Michael Mohler and Rada Mihalcea. 2009. Text-to-text semantic similarity for automatic short answer grading. In *Proceedings of the 12th Conference of the European Chapter of the ACL*.
- Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. 2022. Robust speech recognition via large-scale weak supervision. *arXiv preprint arXiv:2212.04356*.
- Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. [SQuAD: 100,000+ questions for machine comprehension of text](#). In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*.
- Nils Reimers and Iryna Gurevych. 2019. [Sentence-BERT: Sentence embeddings using Siamese BERT-networks](#). In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*.
- Md Arafat Sultan, Cristobal Salazar, and Tamara Sumner. 2016. Fast and easy short answer grading with high accuracy. In *Proceedings of NAACL-HLT*.

A Code and Data Availability

The full source code, evaluation data, and instructions for reproducing all results (including the fine-tuning script) are available at: <https://github.com/Jalalimoghaddam/spoken-rc-eval>.